Close

```
# lab#2 task1
```

# lab#2 task1

```
class MyClass:
    x = 10
    y = 20

obj = MyClass()
print("1 - Namespace:", obj.__dict__)
#2
def student_data(student_id, name=None, student_class=None):
    print("2 - Student Data")
    print("Student ID:", student_id)
    if name:
        print("Name:", name)
    if student_class:
        print("Class:", student_class)

student_data(101)
student_data(102, "Ali")
student_data(103, "Sara", "CS")
#3
class Student:
    pass

print("3 - Class Info")
print("Type:", type(Student))
print("Dict Keys:", Student.__dict__.keys())
print("Module:", Student.__module__)
#4
class StudentEmpty:
    pass

class Marks:
    pass

s = StudentEmpty()
m = Marks()

print("4 - Empty Classes Check")
print("Is s instance of StudentEmpty?", isinstance(s, StudentEmpty))
print("Is m instance of Marks?", isinstance(m, Marks))
print("Is StudentEmpty subclass of object?", issubclass(StudentEmpty, object))
print("Is Marks subclass of object?", issubclass(Marks, object))
#5
class StudentMarks:
```

```
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

sm = StudentMarks("Ali", 85)
print(" 5 - Original:", sm.name, sm.marks)
sm.marks = 90
print("5 - Updated:", sm.name, sm.marks)
#6
class StudentAttr:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

sa = StudentAttr(101, "Ali")
sa.student_class = "CS"   # new attribute
print(" 6 - With new attribute:", sa.__dict__)
del sa.name               # remove attribute
print("6 - After removing name:", sa.__dict__)
#7
class StudentExtended:
    def __init__(self, student_id, name, student_class):
        self.student_id = student_id
        self.name = name
        self.student_class = student_class

    def display(self):
        print(" 7 - Display Attributes")
        for key, value in self.__dict__.items():
            print(key, ":", value)

se = StudentExtended(101, "Ali", "CS")
se.display()
#8
class StudentStruct:
    def __init__(self, student_id, name, student_class):
        self.student_id = student_id
        self.name = name
        self.student_class = student_class

student1 = StudentStruct(101, "Ali", "CS")
student2 = StudentStruct(102, "Sara", "IT")

print("8 - Two Instances")
for s in [student1, student2]:
    print(f"ID: {s.student_id}, Name: {s.name}, Class: {s.student_class}")
 #9
import sys

class StudentMem:
    def __init__(self, student_id, name):
```

```
        self.student_id = student_id
        self.name = name

smem = StudentMem(101, "Ali")
print("9 - Memory size of attributes:", sys.getsizeof(smem.__dict__))
#10
class StudentSlots:
    __slots__ = ['student_id', 'name']  # restrict attributes

    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name

sslots = StudentSlots(101, "Ali")
print(" 10 - Using __slots__:", sslots.student_id, sslots.name)
```

```
1 - Namespace: {}
2 - Student Data
Student ID: 101
2 - Student Data
Student ID: 102
Name: Ali
2 - Student Data
Student ID: 103
Name: Sara
Class: CS
3 - Class Info
Type: <class 'type'>
Dict Keys: dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
Module: __main__
4 - Empty Classes Check
Is s instance of StudentEmpty? True
Is m instance of Marks? True
Is StudentEmpty subclass of object? True
Is Marks subclass of object? True
 5 - Original: Ali 85
5 - Updated: Ali 90
 6 - With new attribute: {'student_id': 101, 'name': 'Ali', 'student_class': 'CS
6 - After removing name: {'student_id': 101, 'student_class': 'CS'}
 7 - Display Attributes
student_id : 101
name : Ali
student_class : CS
8 - Two Instances
ID: 101, Name: Ali, Class: CS
ID: 102, Name: Sara, Class: IT
9 - Memory size of attributes: 296
 10 - Using __slots__: 101 Ali
```

# lab task 2

```
#1
class Trapezoid:
    def __init__(self, base1, base2, height):
        self.base1 = base1
        self.base2 = base2
        self.height = height

    def area(self):
        # Formula: (1/2) * (base1 + base2) * height
        return 0.5 * (self.base1 + self.base2) * self.height


class Parallelogram:
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        # Formula: base * height
        return self.base * self.height


#2 Comparison function (bahar hona chahiye)
def compare_areas(shape1, shape2):
    if shape1.area() > shape2.area():
        return f"{type(shape1).__name__} has larger area"
    elif shape1.area() < shape2.area():
        return f"{type(shape2).__name__} has larger area"
    else:
        return "Both shapes have equal area"


#3 Dictionary for areas
areas_dict = {}

# Example dimensions
t1 = Trapezoid(10, 6, 5)
p1 = Parallelogram(8, 5)

areas_dict["Trapezoid1"] = t1.area()
areas_dict["Parallelogram1"] = p1.area()

print("Task 3 - Areas Dictionary:", areas_dict)

#4 Multiple sets of dimensions
dimensions = [
    {"base1": 10, "base2": 6, "height": 5, "pbase": 8, "pheight": 5},
    {"base1": 12, "base2": 7, "height": 6, "pbase": 10, "pheight": 4},
    {"base1": 8, "base2": 5, "height": 4, "pbase": 7, "pheight": 3}
]
```

```
print("\nTask 4 - Iterating over dimensions")
for i, d in enumerate(dimensions, start=1):
    t = Trapezoid(d["base1"], d["base2"], d["height"])
    p = Parallelogram(d["pbase"], d["pheight"])
    areas_dict[f"Set{i}_Trapezoid"] = t.area()
    areas_dict[f"Set{i}_Parallelogram"] = p.area()
    print(f"Set {i}: Trapezoid Area = {t.area()}, Parallelogram Area = {p.area(
    print("Comparison:", compare_areas(t, p))

#5 Find maximum area
max_shape = max(areas_dict, key=areas_dict.get)
print("\nTask 5 - Shape with Maximum Area:", max_shape, "=", areas_dict[max_sha
```

```
Task 3 - Areas Dictionary: {'Trapezoid1': 40.0, 'Parallelogram1': 40}

Task 4 - Iterating over dimensions
Set 1: Trapezoid Area = 40.0, Parallelogram Area = 40
Comparison: Both shapes have equal area
Set 2: Trapezoid Area = 57.0, Parallelogram Area = 40
Comparison: Trapezoid has larger area
Set 3: Trapezoid Area = 26.0, Parallelogram Area = 21
Comparison: Trapezoid has larger area

Task 5 - Shape with Maximum Area: Set2_Trapezoid = 57.0
```